# LLM 스마트 라우팅

## 개요

요청 종류별 분기 + 부하 기반 로드밸런싱을 구현합니다.

## 전체 트래픽 흐름

```
                                External Traffic
                                       │
                                       ▼
┌──────────────────────────────────────────────────────────────────────┐
│                        Edge Layer (앞단)                              │
│  ┌──────────────────────────────────────────────────────────────────┐ │
│  │                     Ingress Controller (nginx)                   │ │
│  │  - SSL/TLS 종료                                                  │ │
│  │  - 기본 라우팅 (호스트/경로 기반)                                │ │
│  │  - Rate Limiting (기본)                                          │ │
│  │  - IP 화이트리스트/블랙리스트                                    │ │
│  └──────────────────────────────────────────────────────────────────┘ │
└──────────────────────────────────────────────────────────────────────┘
                                       │
                  ┌────────────────────┼────────────────────┐
                  │                    │                    │
                  ▼                    ▼                    ▼
        ┌──────────────────┐   ┌──────────────────┐   ┌──────────────────┐
        │  api.platform/   │   │ admin.platform/  │   │ *.rpc.platform/  │
        │      v1/*        │   │                  │   │                  │
        └──────────────────┘   └──────────────────┘   └──────────────────┘
                  │                    │                    │
                  ▼                    ▼                    ▼
┌──────────────────────────┐  ┌──────────────────┐  ┌──────────────────────┐
│      LLM Router          │  │    Admin API     │  │   Blockchain Direct  │
│  (shared-services ns)    │  │                  │  │   (customer ns로 직접) │
│                          │  │  - 고객 관리     │  │                      │
│  - API 키 인증           │  │  - 모델 배포     │  │  - Cosmos RPC        │
│  - 요청 분류             │  │  - 모니터링      │  │  - ETH JSON-RPC      │
│  - 고객별 라우팅         │  │                  │  │  - WebSocket         │
│  - 부하 기반 선택        │  │                  │  │                      │
└──────────────────────────┘  └──────────────────┘  └──────────────────────┘
                  │
                  ▼
┌──────────────────────────────────────────────────────────────────────┐
│                        Customer Namespaces                            │
│  ┌──────────────────┐  ┌──────────────────┐  ┌──────────────────┐    │
│  │ customer-alpha   │  │ customer-beta    │  │ customer-gamma   │ ...│
│  │  ┌────────────┐   │  │  ┌────────────┐   │  │  ┌────────────┐   │    │
│  │  │vllm-7b     │   │  │  │vllm-13b    │   │  │  │vllm-70b    │   │    │
│  │  │vllm-13b    │   │  │  │tgi-mixtral│   │  │  │            │   │    │
│  │  └────────────┘   │  │  └────────────┘   │  │  └────────────┘   │    │
```

```
│  └──────────────────┘  └──────────────────┘  └──────────────────┘          │
└──────────────────────────────────────────────────────────────────────────────┘
```

## Layer 1: Ingress Controller (앞단 라우터)

### nginx Ingress 설치 및 설정

```yaml
# Helm으로 설치
helm repo add ingress-nginx https://kubernetes.github.io/ingress-nginx
helm repo update

helm install ingress-nginx ingress-nginx/ingress-nginx \
  --namespace ingress-nginx \
  --create-namespace \
  --set controller.service.type=LoadBalancer \
  --set controller.config.proxy-body-size="100m" \
  --set controller.config.proxy-read-timeout="300" \
  --set controller.config.proxy-send-timeout="300"
```

### 글로벌 Rate Limiting

```yaml
# manifests/ingress/configmap.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: ingress-nginx-controller
  namespace: ingress-nginx
data:
  # 전역 설정
  proxy-body-size: "100m"
  proxy-read-timeout: "300"
  proxy-send-timeout: "300"

  # Rate limiting (전역)
  limit-req-status-code: "429"
  limit-conn-status-code: "429"
```

### SSL/TLS 설정 (cert-manager)

```yaml
# manifests/ingress/cluster-issuer.yaml
apiVersion: cert-manager.io/v1
kind: ClusterIssuer
metadata:
  name: letsencrypt-prod
spec:
  acme:
```

```
    server: https://acme-v02.api.letsencrypt.org/directory
    email: admin@yourplatform.io
    privateKeySecretRef:
      name: letsencrypt-prod
    solvers:
    - http01:
        ingress:
          class: nginx
```

### 메인 Ingress 설정

```
# manifests/ingress/main-ingress.yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: platform-main
  namespace: shared-services
  annotations:
    cert-manager.io/cluster-issuer: "letsencrypt-prod"
    nginx.ingress.kubernetes.io/ssl-redirect: "true"
    # Rate limiting
    nginx.ingress.kubernetes.io/limit-rps: "100"
    nginx.ingress.kubernetes.io/limit-connections: "50"
    # Timeouts (LLM 응답 대기)
    nginx.ingress.kubernetes.io/proxy-read-timeout: "300"
    nginx.ingress.kubernetes.io/proxy-send-timeout: "300"
    nginx.ingress.kubernetes.io/proxy-body-size: "100m"
spec:
  ingressClassName: nginx
  tls:
  - hosts:
    - api.yourplatform.io
    - admin.yourplatform.io
    secretName: platform-tls
  rules:
  # LLM API
  - host: api.yourplatform.io
    http:
      paths:
      - path: /v1
        pathType: Prefix
        backend:
          service:
            name: llm-router
            port:
              number: 80
  # Admin API
  - host: admin.yourplatform.io
    http:
      paths:
      - path: /
```

```
        pathType: Prefix
        backend:
          service:
            name: admin-api
            port:
              number: 80
```

### 블록체인 RPC Ingress (고객별)

```yaml
# manifests/ingress/blockchain-ingress-template.yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: blockchain-rpc
  namespace: customer-${CUSTOMER_ID}
  annotations:
    nginx.ingress.kubernetes.io/proxy-read-timeout: "3600"
    nginx.ingress.kubernetes.io/proxy-send-timeout: "3600"
    # WebSocket 지원
    nginx.ingress.kubernetes.io/proxy-http-version: "1.1"
    nginx.ingress.kubernetes.io/upstream-hash-by: "$remote_addr"
spec:
  ingressClassName: nginx
  tls:
  - hosts:
    - ${CUSTOMER_ID}.rpc.yourplatform.io
    secretName: ${CUSTOMER_ID}-rpc-tls
  rules:
  - host: ${CUSTOMER_ID}.rpc.yourplatform.io
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: cosmos-node  # 또는 geth-node
            port:
              number: 26657    # 또는 8545
```

## Layer 2: LLM Router (스마트 라우팅)

### 아키텍처

```
┌──────────────────────────────────────────────────────────────────┐
│                        LLM Router                                 │
│                                                                   │
│  ┌──────────────┐     ┌──────────────┐     ┌──────────────┐     │
│  │   API Key    │     │   Request    │     │    Load      │     │
│  │   Validator  │ -> │  Classifier  │ -> │   Balancer   │     │
```

```
│  │              │    │              │    │              │   │
│  │ - 키 검증    │    │ - 태스크 분류│    │ - 메트릭 조회│   │
│  │ - 고객 식별  │    │ - 모델 매칭  │    │ - 엔드포인트 │   │
│  │ - 쿼터 확인  │    │              │    │   선택       │   │
│  └──────────────┘    └──────────────┘    └──────────────┘   │
│                                                  │          │
│  ┌───────────────────────────────────────────────┼──────┐   │
│  │                Reverse Proxy                  ▼      │   │
│  │  - 요청 전달                                         │   │
│  │  - 스트리밍 응답 처리                                │   │
│  │  - 에러 핸들링                                       │   │
│  │  - 메트릭 수집                                       │   │
│  └──────────────────────────────────────────────────────┘   │
└──────────────────────────────────────────────────────────────┘
```

## 요청 분류 로직

```go
// 태스크 타입 정의
type TaskType string

const (
    TaskCoding   TaskType = "coding"    // 코드 생성/디버깅
    TaskAnalysis TaskType = "analysis"  // 분석/요약
    TaskCreative TaskType = "creative"  // 창작/글쓰기
    TaskChat     TaskType = "chat"      // 일반 대화
    TaskSimple   TaskType = "simple"    // 간단한 질문
)

// 분류 기준
// 1. 명시적 모델 지정 → 해당 모델로 직접 라우팅
// 2. 키워드 기반 분류 → 태스크에 맞는 모델 풀에서 선택
// 3. 토큰 길이 기반 → 짧은 요청은 경량 모델로

func (c *Classifier) Classify(req ClassifyRequest) TaskType {
    // 명시적 모델 지정 시 분류 스킵
    if req.Model != "" && req.Model != "auto" {
        return TaskChat // 기본값, 실제로는 모델로 직접 라우팅
    }

    content := extractUserContent(req.Messages)

    // 짧은 요청
    if len(strings.Fields(content)) < 10 {
        return TaskSimple
    }

    // 키워드 매칭
    scores := map[TaskType]int{
        TaskCoding:   countMatches(content, codingKeywords),
        TaskAnalysis: countMatches(content, analysisKeywords),
        TaskCreative: countMatches(content, creativeKeywords),
```

```
    }

    // 최고 점수 태스크 반환
    return getMaxScoreTask(scores)
}
```

### 부하 기반 로드밸런싱

```
type EndpointSelector struct {
    prometheusClient *prometheus.Client
    cache            *EndpointCache
}

func (s *EndpointSelector) SelectEndpoint(
    ctx context.Context,
    namespace string,
    task TaskType,
    candidates []ModelEndpoint,
) (*ModelEndpoint, error) {

    var scored []ScoredEndpoint

    for _, ep := range candidates {
        // 프로메테우스에서 실시간 메트릭 조회
        metrics := s.getEndpointMetrics(ctx, namespace, ep.Name)

        // 점수 계산 (낮을수록 좋음)
        score := calculateScore(metrics)

        scored = append(scored, ScoredEndpoint{
            Endpoint: ep,
            Score:    score,
        })
    }

    // 점수순 정렬
    sort.Slice(scored, func(i, j int) bool {
        return scored[i].Score < scored[j].Score
    })

    // Weighted random selection (완전 최저만 선택하면 thundering herd)
    return weightedSelect(scored), nil
}

func calculateScore(m EndpointMetrics) float64 {
    // 가중치 조정 가능
    return m.QueueLength*3.0 +      // 큐 길이 (가장 중요)
           m.GPUUtilization/100.0 + // GPU 사용률
           m.P99Latency/10.0        // 지연 시간
}
```

### 모델-태스크 매핑 설정

```yaml
# configmaps/routing-config.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: routing-config
  namespace: shared-services
data:
  routing.yaml: |
    # 태스크별 권장 모델 (우선순위 순)
    task_models:
      coding:
        - name: codellama-34b
          priority: 1
        - name: deepseek-coder-33b
          priority: 2
        - name: llama2-70b
          priority: 3  # fallback

      analysis:
        - name: llama2-70b
          priority: 1
        - name: mixtral-8x7b
          priority: 2
        - name: qwen-72b
          priority: 3

      creative:
        - name: llama2-70b
          priority: 1
        - name: mixtral-8x7b
          priority: 2

      chat:
        - name: llama2-13b
          priority: 1
        - name: mistral-7b
          priority: 2
        - name: llama2-7b
          priority: 3

      simple:
        - name: phi-2
          priority: 1
        - name: mistral-7b
          priority: 2
        - name: llama2-7b
          priority: 3

    # 모델 스펙
    models:
```

```
      llama2-7b:
        max_context: 4096
        capabilities: [chat, simple]
        cost_tier: low

      llama2-13b:
        max_context: 4096
        capabilities: [chat, analysis]
        cost_tier: medium

      llama2-70b:
        max_context: 4096
        capabilities: [coding, analysis, creative, chat]
        cost_tier: high

      codellama-34b:
        max_context: 16384
        capabilities: [coding]
        cost_tier: high

      mixtral-8x7b:
        max_context: 32768
        capabilities: [analysis, creative, chat]
        cost_tier: high
```

## Layer 3: API Gateway 기능 (선택적)

더 고급 기능이 필요하면 Kong, APISIX, 또는 Envoy Gateway 추가 가능:

### Kong Gateway 예시

```yaml
# manifests/gateway/kong-plugin-ratelimit.yaml
apiVersion: configuration.konghq.com/v1
kind: KongPlugin
metadata:
  name: rate-limit-by-customer
  namespace: shared-services
config:
  minute: 1000
  policy: redis
  redis_host: redis.shared-services.svc
plugin: rate-limiting

---
apiVersion: configuration.konghq.com/v1
kind: KongPlugin
metadata:
  name: key-auth
  namespace: shared-services
plugin: key-auth
```

자체 구현 시 추가할 기능

```go
// middleware/auth.go
func APIKeyAuthMiddleware(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        apiKey := extractAPIKey(r)

        // 1. API 키 검증
        customer, err := validateAPIKey(apiKey)
        if err != nil {
            http.Error(w, "Invalid API key", http.StatusUnauthorized)
            return
        }

        // 2. 쿼터 확인
        if !checkQuota(customer) {
            http.Error(w, "Quota exceeded", http.StatusTooManyRequests)
            return
        }

        // 3. Rate limiting
        if !rateLimiter.Allow(customer.ID) {
            http.Error(w, "Rate limit exceeded", http.StatusTooManyRequests)
            return
        }

        // 컨텍스트에 고객 정보 추가
        ctx := context.WithValue(r.Context(), "customer", customer)
        next.ServeHTTP(w, r.WithContext(ctx))
    })
}
```

## 스트리밍 응답 처리

LLM의 스트리밍 응답(SSE)을 올바르게 프록시:

```go
// proxy/streaming.go
func (p *Proxy) handleStreamingResponse(w http.ResponseWriter, r *http.Request,
target *url.URL) {
    // Flush 지원 확인
    flusher, ok := w.(http.Flusher)
    if !ok {
        http.Error(w, "Streaming not supported", http.StatusInternalServerError)
        return
    }

    // 헤더 설정
    w.Header().Set("Content-Type", "text/event-stream")
    w.Header().Set("Cache-Control", "no-cache")
    w.Header().Set("Connection", "keep-alive")
```

```go
    w.Header().Set("X-Accel-Buffering", "no") // nginx 버퍼링 비활성화

    // 업스트림 요청
    resp, err := p.client.Do(proxyReq)
    if err != nil {
        // 에러 처리
        return
    }
    defer resp.Body.Close()

    // 스트리밍 복사
    reader := bufio.NewReader(resp.Body)
    for {
        line, err := reader.ReadBytes('\n')
        if err != nil {
            if err == io.EOF {
                break
            }
            return
        }

        w.Write(line)
        flusher.Flush()
    }
}
```

## 모니터링 메트릭

```yaml
# prometheus-rules/router-alerts.yaml
groups:
- name: router-alerts
  rules:
  # 라우터 지연 시간
  - alert: RouterHighLatency
    expr: |
      histogram_quantile(0.99,
        rate(llm_router_routing_latency_seconds_bucket[5m])
      ) > 0.1
    for: 5m
    labels:
      severity: warning
    annotations:
      summary: "Router P99 latency > 100ms"

  # 엔드포인트 선택 실패
  - alert: NoAvailableEndpoints
    expr: |
      increase(llm_router_no_endpoint_errors_total[5m]) > 10
    labels:
      severity: critical
    annotations:
```

```
      summary: "No available endpoints for routing"

  # Rate limit 초과 빈도
  - alert: HighRateLimitHits
    expr: |
      sum(rate(llm_router_rate_limit_hits_total[5m])) by (namespace) > 100
    labels:
      severity: warning
    annotations:
      summary: "High rate limit hits for {{ $labels.namespace }}"
```

## 정리: 라우팅 레이어별 역할

| Layer | 컴포넌트 | 역할 | 필수 여부 |
|---|---|---|---|
| Edge | Ingress Controller (nginx) | SSL 종료, 기본 라우팅, Rate Limit | ☑ 필수 |
| Application | LLM Router | API 키 인증, 요청 분류, 부하 분산 | ☑ 필수 |
| (Optional) | API Gateway | 고급 Rate Limit, 쿼터 관리, 분석 | 선택적 |
| (Optional) | Service Mesh | 트래픽 미러링, 카나리, 서킷브레이커 | 선택적 |

**현재 구현**: Edge + Application 레이어 (대부분의 케이스에 충분)

다음: 07-AIOPS-INTEGRATION.md